STAGESYNC: A COLLABORATIVE 3D STAGE-PLANNING AND PRODUCTION COORDINATION PLATFORM

PROJECT REPORT

Submitted by

ABHISHEK R	**-**	**241701001**
HARINI M	**-**	**241701017**

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING IN COMPUTER SCIENCE AND DESIGN



RAJALAKSHMI ENGINEERING COLLEGE

ANNA UNIVERSITY, CHENNAI

OCTOBER 2026

BONAFIDE CERTIFICATE

Certified that this Report titled "STAGESYNC: A COLLABORATIVE 3D STAGE-PLANNING AND PRODUCTION COORDINATION PLATFORM" is the bonafide work of ABHISHEK R (241701001) and HARINI M (241701017) who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

Certified further that the project contributes to SDG 9 – Industry, Innovation and Infrastructure and SDG 4 – Quality Education, with a primary focus on SDG 9 through a collaborative, browser-based digital platform for event-production planning.

Signature of the HOD

Dr. S. Vinodh Kumar
Professor, The Head of the Department
Department of Computer Science and Design
Rajalakshmi Engineering College
Chennai

Signature of the Supervisor

Dr. P. Revathy
Professor
Department of Computer Science and Design
Rajalakshmi Engineering College
Chennai

Submitted for the Phase I Project Viva-Voce examination held on _____.

INTERNAL EXAMINER **EXTERNAL EXAMINER**

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman Mr. S. MEGANATHAN, B.E, F.I.E., our Vice Chairman Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S., and our respected Chairperson Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D., for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to Dr. S.N. MURUGESAN, M.E., Ph.D., our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to Dr. S. VINODHKUMAR, Ph.D., Professor and Head of the Department of Computer Science and Design for his guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide and faculty incharge, Dr. P. REVATHY, Department of Computer Science and Design, for her valuable guidance throughout the course of the project.

We also thank all the faculty members and staff of the Department of Computer Science and Design, Rajalakshmi Engineering College, and our fellow students who shared their experience of organizing college events, which shaped the design challenge addressed in this work.

ABSTRACT

Live events, cultural programmes and theatrical productions depend on close coordination between stage design, props, lighting and sound teams. In practice, these teams often work through separate messages, documents, sketches and verbal discussions, which leads to unclear responsibilities, late changes and spatial clashes that surface only during set-up. StageSync is a proposed cloud-native collaborative 3D stage-planning and production-coordination platform that brings these teams into one shared, browser-based 3D scene. The design combines React, TypeScript, Three.js and React Three Fiber on the client with a Supabase/PostgreSQL back end, a role-based access control model that maps departmental privileges to scene changes, an optimistic concurrency control protocol for simultaneous edits, and an Art Director review-and-approval workflow tied to a production timeline. A venue-aware AI pipeline, planned as a later feature, would convert a venue photograph and user-supplied dimensions into an editable parametric stage layout.

The report documents the design challenge framed through the S.C.O.P.E.S sheet, the literature background, system requirements, architecture, data-flow, use-case and activity diagrams, module and data design, and a structured test plan. The implementation status is stated conservatively: the architecture, requirements and test plan are defined. Planned evaluation covers collaboration turnaround time, revision cycles, conflict detection, state-synchronization latency and viewport frame rate. No measured performance results are claimed in this report. StageSync is intended as a planning aid for event teams and not as a substitute for certified structural, rigging or safety engineering.

Keywords: collaborative 3D planning, stage design, event production, Three.js, React Three Fiber, optimistic concurrency control, role-based access control, Supabase, venue-aware AI.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
Chapter 1	INTRODUCTION	1
Chapter 2	LITERATURE REVIEW AND EXISTING SYSTEMS	5
Chapter 3	SYSTEM ANALYSIS AND DESIGN	8
Chapter 4	IMPLEMENTATION METHODOLOGY AND TESTING	18
Chapter 5	CONCLUSION AND FUTURE ENHANCEMENTS	24
	REFERENCES	26
Appendix A	TOOLS AND ENVIRONMENT CHECKLIST	28
Appendix B	MUTATION EXCHANGE TRACE TEMPLATE	29
Appendix C	S.C.O.P.E.S TEAM SCOPE SHEET SUMMARY	30

LIST OF FIGURES

Figure Number	Figure Name	Page Number
Figure 3.1	Overall StageSync system architecture	9
Figure 3.2	Data Flow Diagram — Level 0 (Context)	10
Figure 3.3	Data Flow Diagram — Level 1	10
Figure 3.4	Data Flow Diagram — Level 2 (Mutation Processing)	11
Figure 3.5	StageSync use-case diagram	11
Figure 3.6	Activity diagram — Scene edit and review lifecycle	12
Figure 3.7	Overall StageSync functional diagram	12
Figure 3.8	Core conceptual classes / entities	16
Figure 3.9	Venue-aware AI stage generation pipeline	17
Figure 4.1	Governance state machine for stage objects	20
Figure 4.2	Approval page dashboard	20
Figure 4.3	3D stage editor with properties inspector	21

LIST OF TABLES

Table Number	Table Name	Page Number
Table 1.1	Limitations of conventional event-planning approaches	3
Table 2.1	Literature and technology comparison	6
Table 2.2	Comparative analysis of stage-planning approaches	7
Table 3.1	Functional requirements	8
Table 3.2	Non-functional requirements	8
Table 3.3	Development resource inventory	13
Table 3.4	Role-based access control permission matrix	13
Table 3.5	Software technology stack	14
Table 3.6	System module descriptions	14
Table 3.7	Scene mutation message structure	15
Table 3.8	Conceptual data entities	15
Table 4.1	Development and validation stages	18
Table 4.2	Known development status	18
Table 4.3	Test cases and current status	21
Table 4.4	Performance metrics	21
Table 4.5	Performance measurement template	22
Table 5.1	Future enhancement plan	24
Table 5.2	Proposed Phase-II work schedule	25
Table A.1	Tools and environment checklist	28
Table B.1	Mutation exchange trace template	29
Table C.1	SCOPES summary of the design challenge	30

CHAPTER 1 — INTRODUCTION

1.1 General

Live event production is a collaborative, multidisciplinary activity carried out under fixed deadlines. Whether the occasion is a college cultural programme, a theatrical performance, a concert or a corporate conference, the stage must be designed, built, dressed, lit and wired for sound by several teams whose work physically overlaps in the same space. Stage design, props, lighting and sound teams each hold part of the plan, and the Art Director is responsible for the overall visual result.

StageSync investigates a shared, browser-based 3D environment in which these teams place and edit stage elements in one scene, record requirements against the objects they concern, and route their changes through an Art Director review and approval step. Rather than relying on separate chat messages, documents and sketches, the proposed design keeps one authoritative copy of the stage plan that every team can see.

1.2 Background

Conventional stage planning relies on desktop CAD and visualization packages, static two-dimensional technical drawings, spreadsheets and messaging applications [18]. Desktop tools provide precise drafting but are designed for a single operator at a workstation, so a change made by one department reaches the others only when a file is re-exported and re-circulated. In student and community events, the situation is usually simpler but no better: plans live in sketches, group chats and verbal agreements.

Modern browsers can render interactive 3D graphics through WebGL, and libraries such as Three.js [1] and React Three Fiber [2] make it practical to build a 3D editor as an ordinary web application. Managed back-end services such as Supabase [4], built on PostgreSQL [5], provide authentication, row-level security and real-time change broadcasting. StageSync uses these building blocks to separate the 3D interface from the authoritative shared state, so that several users can work on the same scene while each department is restricted to its own responsibilities.

1.3 Problem Statement

During the planning of events, teams handling stage design, props, lighting and sound often communicate through separate messages, documents, sketches and verbal discussions. This leads to miscommunication, unclear responsibilities and last-minute changes. The problem commonly occurs in college events, cultural programmes and other live productions involving multiple teams [18].

The design challenge, framed through the S.C.O.P.E.S sheet (Appendix C), is: **How might we improve coordination between different event production teams and enable the Art Director to effectively review, provide feedback and approve stage-related**

requirements before the event? The challenge is stated as an outcome rather than a solution. StageSync is the team's proposed answer to it, and it must be evaluated against that outcome.

1.4 Motivation

The project is motivated by recurring difficulties observed in event preparation: separate communication channels for each team, unclear requirements and ownership, frequent last-minute changes, and difficulty reviewing the complete stage plan in one place [18]. Short preparation timelines leave little room to resolve conflicts once they surface, and non-technical stakeholders find it hard to interpret two-dimensional schematics.

The stakeholders identified in the scope sheet are summarized below. Primary stakeholders are the Art Director, Event Organizer, Props and Stage Design team, Lighting team and Sound team. Secondary stakeholders are Stage Managers, Event Coordinators and the Production Crew. The decision makers are the Art Director and the Event Head. A shared, visual plan with clear ownership is expected to reduce time spent chasing updates and to let the Art Director approve requirements with confidence.

1.5 Aim

To design and develop a collaborative, browser-based 3D stage-planning and production-coordination platform that centralizes stage requirements, clarifies departmental responsibilities, and supports Art Director review and approval before an event.

1.6 Objectives

- Centralize requirements from the stage, props, lighting and sound teams in one shared, visual plan.
- Clarify ownership so that each team knows what is expected of it and by when, using role-based access control.
- Reduce last-minute rework by surfacing changes to affected teams as soon as they happen.
- Provide the Art Director with a clear review, annotation and approval workflow for stage-related changes.
- Provide a shared timeline view so that every team can see what happens at each point of the event.
- Handle simultaneous edits safely using optimistic concurrency control with a bounded locking fallback.
- Provide a reusable library of 3D props and staging components in the glTF format.
- Investigate a venue-aware AI pipeline that proposes an initial stage layout from a venue photograph and dimensions.
- Define an evaluation method for planning time, revision cycles, conflicts detected and synchronization latency, and document security and privacy constraints.

1.7 Scope

1.7.1 Included Scope

- A web-based 3D scene editor with transform controls, grid snapping and basic collision warnings.
- Account-based sign-in and role-based restriction of scene changes by department.
- A properties inspector, a reusable asset library and a shared timeline view.
- An Art Director review state (pending, approved, revision required) attached to scene objects.
- Real-time synchronization of scene changes between connected users.
- A planned, optional AI-assisted layout feature taking a venue photograph and dimensions as input.
- A defined test and evaluation plan using free or low-cost tools suitable for a student project.

1.7.2 Scope Limitations

- The behaviour of each module must be confirmed against the source code and tests; this report does not claim that every designed feature is already working.
- No measured performance, usability or scalability results are reported. All figures in the evaluation chapter are plans or templates.
- The AI layout feature is a proposed design; its accuracy on real venue photographs has not been measured.
- Structural, rigging-load and electrical safety calculations are outside the scope; StageSync does not replace certified engineering review.
- Offline operation, mobile optimization and large-scale deployment are not assumed.

1.8 Existing System and Limitations

Table 1.1 Limitations of conventional event-planning approaches

Approach	Strength	Limitation relevant to this project
Messaging apps and e-mail	Fast, familiar to every team	Requirements scattered across threads; no single plan; changes easy to miss
Documents and spreadsheets	Easy task and requirement lists	No spatial context; cannot show whether two items collide
Hand sketches and 2D plans	Quick to produce; good for ideas	Hard for non-technical stakeholders to read; versions diverge
Desktop CAD / visualizers	Precise drafting and lighting paperwork	Single-user files; licensing cost; steep learning curve
General cloud design tools	Real-time collaboration on 2D boards	No stage-specific objects, roles or approval workflow

Existing planning approaches differ in cost, precision and collaboration support. Desktop CAD tools offer drafting precision but not simultaneous multi-department editing. General collaboration tools offer shared boards but no 3D spatial awareness. Messaging and

documents are the most common practice in college events, and they leave the Art Director to assemble the picture manually from scattered updates.

1.9 Proposed System

StageSync proposes three cooperating tiers. A browser client renders the 3D stage and applies edits optimistically. A gateway layer authenticates users, enforces role-based permissions, checks object versions for concurrency, and broadcasts accepted changes. A PostgreSQL-based persistence tier stores projects, scene objects, review records, roles and assets. Every scene change that passes validation marks the affected object as pending review, so the Art Director sees what changed, can annotate it, and approves it or requests revision. Approved items feed a timeline view. A venue-aware AI service is planned as an optional extension. This is a proposed architecture; each behaviour must be verified through tests.

1.10 Advantages and Expected Contribution

- Provides one shared source of truth for the stage plan instead of scattered messages and files.
- Makes ownership explicit through department roles and restricted editing.
- Surfaces changes to affected teams and the Art Director as soon as they occur.
- Requires only a web browser, with no installation, and relies mainly on free or low-cost tools.
- Supports non-technical stakeholders through an intuitive 3D view rather than technical drawings.
- Provides a defined basis for measuring whether coordination actually improves.

CHAPTER 2 — LITERATURE REVIEW AND EXISTING SYSTEMS

2.1 Introduction

This chapter summarizes the technology areas that inform StageSync: event-production coordination needs, web-based 3D graphics, concurrency control for shared editing, and AI-based spatial reconstruction from images. The comparison is meant to position the project design and not to claim that StageSync is the first system of its kind.

2.2 Event-Production Coordination Requirements

Stage production brings several specialties into one physical space: stage carpentry and decking, props and set dressing, lighting design, audio reinforcement and creative direction. Each has its own spatial objectives and vocabulary, and a change by one team can invalidate the work of another. A lighting truss moved to avoid an obstruction may block a speaker sightline; a prop placed for visual effect may block a performer's exit. Coordination therefore requires a shared view of positions, a clear record of who owns each element, and a decision point at which the creative lead accepts or rejects changes.

Commercial lighting and stage tools such as Vectorworks Spotlight and CAST WYSIWYG provide detailed rigging and photometric paperwork, but they are desktop applications with proprietary file formats that assume one operator per file. Version control is typically manual, by renaming files, and the licensing model limits access to specialist operators, which excludes event coordinators and organizers who are central to approval [18].

2.3 Web-Based 3D Graphics and Collaborative Environments

WebGL, and increasingly WebGPU, allow interactive 3D rendering inside ordinary browsers. Three.js [1] provides an imperative scene graph, while React Three Fiber [2] reconciles React components into that scene graph, so that each stage element can be a declarative component. The glTF 2.0 format [6] provides a compact runtime format for 3D assets and is widely supported. Browser-based editors remove installation barriers, but they face memory and draw-call limits, which are managed through instanced rendering, level of detail and compressed meshes.

General 3D web tools can model objects, but they do not carry stage-specific metadata such as departments, review status or production phases. Collaborative 2D design tools demonstrate that real-time multi-user editing is feasible in the browser, but extending it to 3D scenes with departmental restrictions and approvals remains a design problem for each application.

2.4 Concurrency Control for Shared Editing

When several users edit the same scene, the system must decide what happens when two edits conflict. Operational Transformation [8] and Conflict-free Replicated Data Types [9] provide strong convergence guarantees for text-like data. For a scene where each object

carries a transform that changes at high frequency, simpler approaches may suffice. Optimistic concurrency control, introduced by Kung and Robinson [7], lets clients proceed without locks and validates at commit time, rejecting a change that was based on an outdated version. This suits scenes in which most edits touch different objects. Where one object is contested repeatedly, a short exclusive lease can prevent endless rejection loops. StageSync adopts this hybrid approach as a design choice that must be tested under realistic contention.

2.5 AI-Assisted Spatial Reconstruction from Images

Monocular depth estimation models such as MiDaS [10] and Dense Prediction Transformers [11] predict relative depth from a single image, and Segment Anything [12] can isolate regions such as the floor. Classical computer-vision methods based on vanishing points and feature matching [13], [14] recover camera geometry. Converting these outputs into clean, editable, metrically scaled stage geometry requires calibration against known dimensions and a placement rule that respects clearances. The depth predictions are relative, so user-provided venue dimensions are needed to fix scale. The accuracy of such a pipeline on real venue photographs, especially wide-angle images, has to be measured rather than assumed.

2.6 Literature and Technology Comparison

Table 2.1 Literature and technology comparison

Technology / source area	Primary relevance	Design implication for StageSync
Three.js [1]; React Three Fiber [2]	Browser 3D rendering and declarative scene components	Build the stage editor as components; manage draw calls and level of detail
Supabase [4]; PostgreSQL [5]	Auth, realtime channels, row-level security	Enforce department permissions in the database, not only in the UI
glTF 2.0 [6]	Runtime 3D asset format	Store the prop library as glTF; validate uploads
Optimistic concurrency [7]	Commit-time conflict detection	Version each scene object and reject stale edits
OT [8] and CRDT [9]	Convergent shared editing	Considered; not adopted as the primary model for transform data
MiDaS [10]; DPT [11]; SAM [12]	Depth and floor segmentation from one image	Candidate components of the planned layout pipeline
SIFT [13]; OpenCV [14]	Feature matching, camera geometry	Possible scale and pose refinement from multiple photos
ISO 9613-2 [15]; ANSI E1.11 [17]	Sound attenuation; DMX512 control	Reference for planned acoustic and lighting aids; not claimed as implemented

2.7 Comparative Analysis

Table 2.2 Comparative analysis of stage-planning approaches

Feature	Chat + documents	Desktop CAD	Proposed StageSync
Shared live view of the plan	No	No (file-based)	Intended: one shared 3D scene
Simultaneous multi-user editing	Not applicable	Generally no	Intended, using version checks
Department-level permissions	Informal	Limited	Intended: role-based, database-enforced
Art Director approval workflow	Ad hoc messages	External	Intended: built-in review states
Spatial conflict awareness	None	Manual inspection	Basic collision warnings planned
Installation required	None	Yes, licensed	None; runs in a browser
Performance evidence	Not applicable	Vendor documentation	Not yet established; to be measured

2.8 Research Gap and Intended Contribution

Existing tools either provide spatial precision without collaboration, or collaboration without spatial awareness. The intended contribution of StageSync is an integrated, student-scale design that places stage-specific 3D editing, department-level permissions, concurrency handling and Art Director approval in one browser application, together with a defined evaluation method. A claim of novelty would require a fuller literature search and must not be inferred merely from combining familiar components.

2.9 Chapter Summary

The reviewed areas support building a browser-based 3D planning tool on Three.js and React Three Fiber, using a relational back end with row-level security, applying optimistic concurrency with a bounded lock fallback, and investigating AI-assisted layout as a later extension. The feasibility of the proposed system depends on correct implementation of permissions and synchronization and on measured results.

CHAPTER 3 — SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

This chapter defines the logical system and the diagrams needed to guide implementation. The diagrams show the intended design; they are not evidence that every component is operational.

3.2 System Requirements

3.2.1 Functional Requirements

Table 3.1 Functional requirements

ID	Functional requirement
FR-01	The system shall allow a registered user to sign in with an account and be assigned a department role.
FR-02	The system shall allow a project to be created with a venue boundary and a stage scene.
FR-03	Users shall be able to add, move, rotate and scale stage objects within the permissions of their role.
FR-04	The system shall reject an edit to an object that falls outside the user’s department scope.
FR-05	Each scene object shall carry a version number used to detect conflicting simultaneous edits.
FR-06	Accepted changes shall be broadcast to all connected users of the project.
FR-07	Every accepted structural change shall set the object’s review status to pending.
FR-08	The Art Director shall be able to approve a change or request revision, with a comment or annotation.
FR-09	The system shall provide a reusable library of 3D props and staging components.
FR-10	The system shall provide a shared timeline view of production phases and milestones.
FR-11	The system should warn when two objects overlap in 3D space.
FR-12	The system may generate an initial stage layout from a venue photograph and dimensions.

3.2.2 Non-Functional Requirements

Table 3.2 Non-functional requirements

ID	Requirement	Design consideration
NFR-01	Usability	Non-technical users must be able to read the 3D view; roles must be clear in the interface.
NFR-02	Responsiveness	Local edits should feel immediate; synchronization latency is to be measured, with a design target to be agreed.
NFR-03	Performance	Maintain an interactive frame rate on typical laptops; use instancing and level of detail.
NFR-04	Security	Enforce permissions in the database; use authenticated, encrypted connections.
NFR-05	Integrity	Version checks must prevent silent overwriting of another user’s change.

ID	Requirement	Design consideration
NFR-06	Maintainability	Separate rendering, state synchronization, permissions and AI components.
NFR-07	Cost	Rely on free or low-cost tools and hosting suitable for a student project.
NFR-08	Privacy	Collect only the information needed; protect unreleased event designs.

3.3 Overall System Architecture

The architecture separates the client presentation tier, the gateway and concurrency tier, and the persistence and storage tier. The client renders the 3D scene and applies edits optimistically. The gateway authenticates users, checks permissions and versions, and broadcasts accepted changes. The persistence tier stores project data in PostgreSQL and 3D assets in object storage. The AI service is optional and is shown dashed.

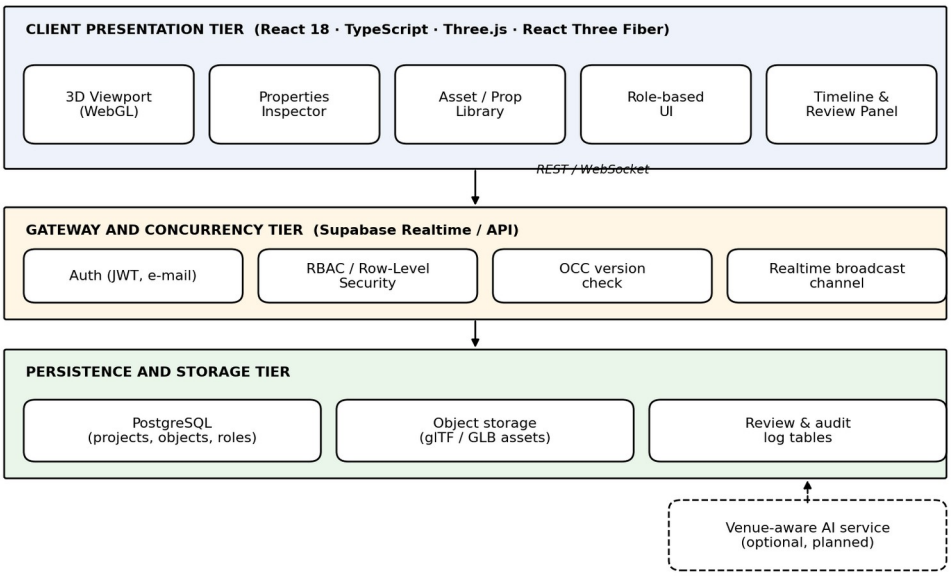


Figure 3.1 Overall StageSync system architecture

3.4 Data Flow Diagram

The DFDs describe the system by showing external entities, processes, data stores and data movement. The stores shown are logical design elements; the actual database schema must be confirmed from the implementation.

3.4.1 DFD Level 0 — Context Diagram

The context diagram treats StageSync as one system. Production team members send scene edits and requirements and receive live updates. The Admin manages users and roles. The Art Director sends review decisions and receives pending reviews. The Event Head and Stage Manager view the plan and timeline. An AI service and asset store are shown as optional external elements.

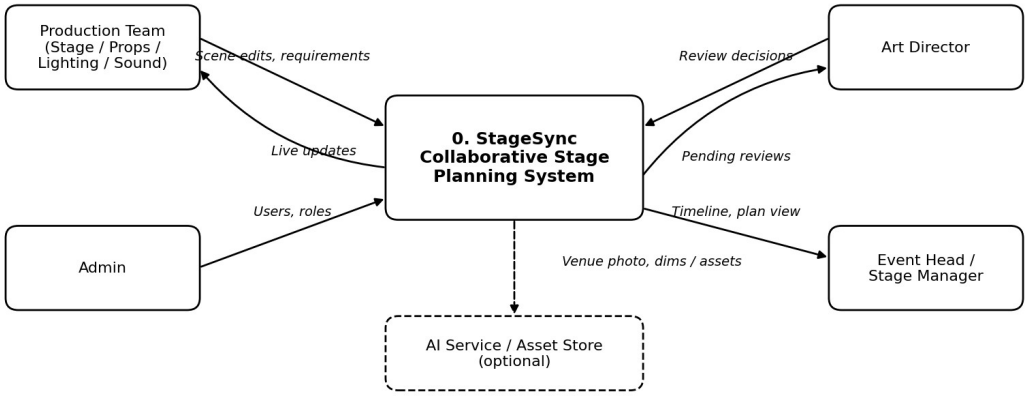


Figure 3.2 Data Flow Diagram — Level 0 (Context)

3.4.2 DFD Level 1

Level 1 decomposes the system into authentication and authorization, scene-object management, concurrency validation, review and approval, timeline and notifications, and the optional AI layout process. The logical stores are users and roles, scene objects, review records and the asset library.

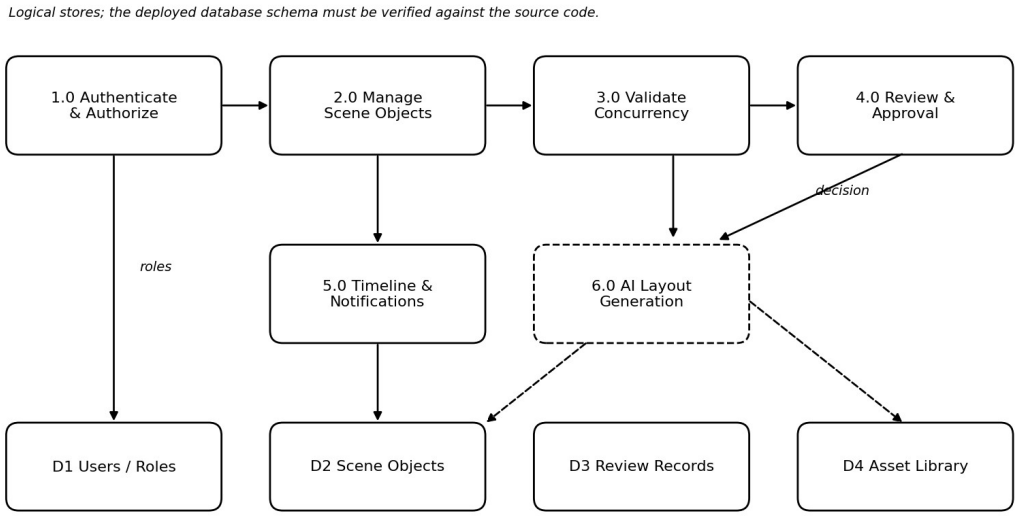


Figure 3.3 Data Flow Diagram — Level 1

3.4.3 DFD Level 2 — Mutation Processing

Level 2 expands the processing of an incoming scene change. The user’s token and role are verified, then the client’s version number is compared with the stored version. A stale version is rejected. A change that passes is checked for spatial overlap, committed with an incremented version and a pending-review status, and broadcast to connected clients. The exact policy must be implemented consistently on the server.

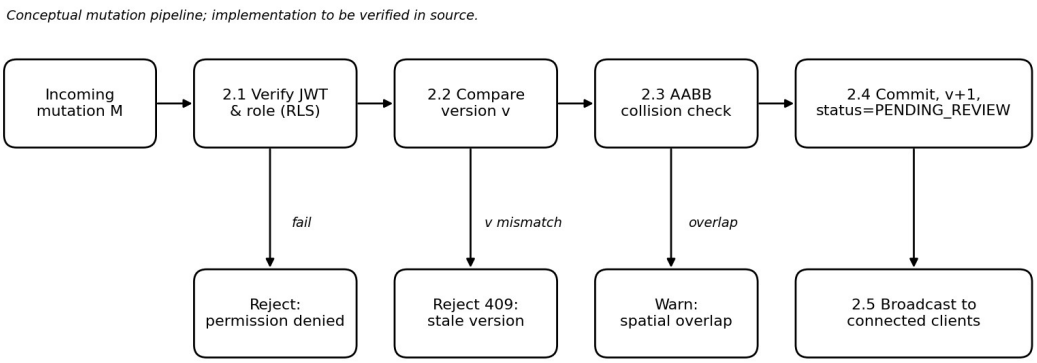


Figure 3.4 Data Flow Diagram — Level 2 (Mutation Processing)

3.5 Use-Case Diagram

The primary actors are the Admin, the Art Director and department members (stage design, props, lighting, sound), with the Event Head viewing plans and timelines. A 3D modeler uploads assets and an AI service supports the optional layout feature. Use cases include managing roles, placing and editing objects, reviewing changes and viewing the timeline. The diagram represents intended use cases rather than proof of implemented features.

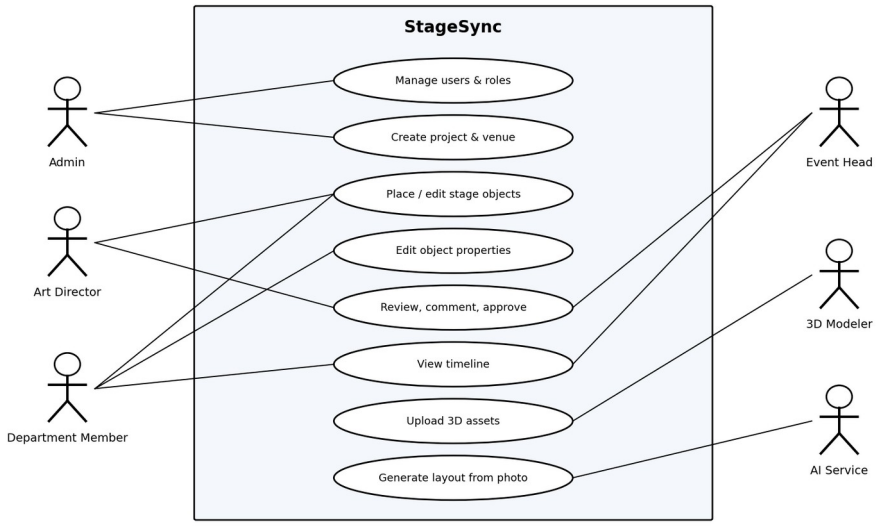


Figure 3.5 StageSync use-case diagram

3.6 Activity Diagram

The activity flow begins when a member signs in and opens a project. A change is applied locally for immediate feedback and sent to the server with the object’s version. The server validates role, version and collision. A rejected change is reverted on the client; an accepted change is committed, marked pending review, and broadcast. The Art Director reviews and either approves or requests revision, and approved items proceed to the production timeline.

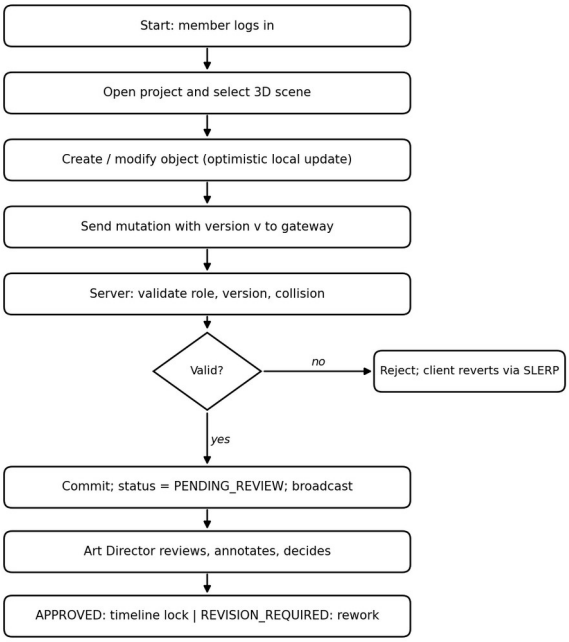


Figure 3.6 Activity diagram — Scene edit and review lifecycle

3.7 Overall System Diagram

The functional diagram groups the design into user, scene, synchronization, governance and data layers, supported by the asset pipeline, the timeline and notification service, and the planned AI layout service. It complements the architecture diagram by emphasizing functional responsibility and separation of concerns.

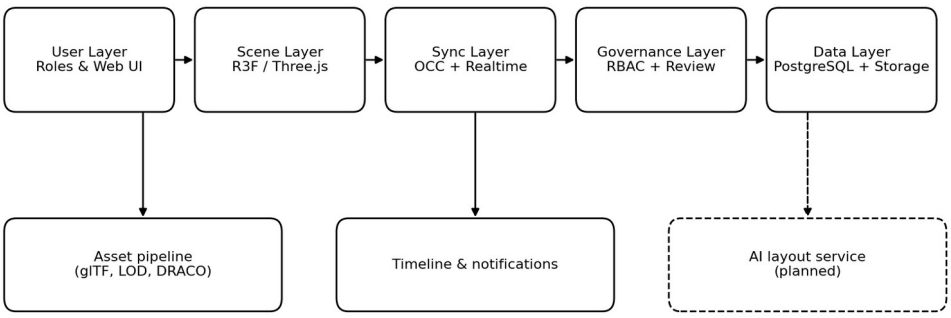


Figure 3.7 Overall StageSync functional diagram

3.8 Development Resource and Role Design

The scope sheet estimates the resources for the project as mainly free or low-cost tools suitable for a student team. Table 3.3 lists the resources known from the scope sheet and design documents; the exact versions in use must be confirmed from the project configuration.

Table 3.3 Development resource inventory

Resource	Purpose	Notes
React / Next.js (free, open source)	Web application front end	Scope-sheet estimate; actual framework to be confirmed from source
TypeScript, Three.js, React Three Fiber	Typed UI code and 3D scene rendering	Core stack of the design
Supabase / PostgreSQL (free tier)	Authentication, realtime, relational storage	Free tier limits apply
Autodesk Maya 2027	Authoring 3D props and stage assets	Exported to glTF for the library
Figma / Miro (free tier)	Visual planning and interface mock-ups	Used for design exploration
Google Calendar API (free)	Possible timeline integration	Optional; not assumed implemented
Cloud hosting (student credits, then paid)	Deployment	Cost to be reviewed before deployment

3.8.1 Role-Based Access Control Design

Each user is assigned a department role, and the database restricts which scene objects that role may change. The intended permissions are summarized in Table 3.4. The number and names of roles in the deployed schema must be checked against this design, since the role list may evolve during implementation.

Table 3.4 Role-based access control permission matrix

Role	Scene mutation scope	Approval authority
Admin	Full read/write; manages users and roles	Project creation, archival
Art Director	Full visual inspection; camera bookmarks; annotations	Approve or request revision
Stage Design	Risers, trusses, proscenium, main framing	Submits layout for review
Props Team	Props, set dressing, furniture	Sign-off on prop list
Lighting Lead	Fixtures and truss clamp targets, beam parameters	Sign-off on lighting plan
Audio / Sound	Speaker stacks, monitors, coverage parameters	Sign-off on sound coverage
3D Modeler	Asset upload, level-of-detail and collision meshes	Asset library validation
Animation	Kinematic rigs and motion cues	Motion-path safety check

3.8.2 Design Constraints

- Permissions must be enforced in the database through row-level security, not only by hiding interface controls.
- A role must be able to view the whole scene but change only objects within its scope.

- Rendering must stay responsive on typical laptops; draw calls are reduced by instancing identical components.
- Hosting and tooling should fit free or low-cost tiers during Phase I and Phase II.

3.8.3 3D Rendering and Scene Design

Each stage element (deck, truss, light, speaker stack, prop) is a React component bound to a Three.js object with a position, rotation and scale. Repeated components such as deck modules and truss segments are rendered with instancing to limit draw calls. Heavy assets use levels of detail, and compressed glTF files are loaded asynchronously so the viewport stays responsive. A grid-snapping option (for example 0.1 m, 0.5 m or 1.0 m) aligns modular decks, and axis-aligned bounding boxes provide basic overlap warnings. Figures for draw-call reduction or frame rate are not claimed here; they are evaluation targets (Section 4.6).

3.9 Software Architecture and Technology Stack

Table 3.5 Software technology stack

Layer / component	Technology	Status / role
Front end	React, TypeScript	Planned/current stack; actual modules to be confirmed from source
3D rendering	Three.js, React Three Fiber	Scene graph and declarative components
Authentication	E-mail sign-in via managed back end	Per-user accounts; role assignment table
Database	PostgreSQL (Supabase)	Projects, objects, roles, reviews
Realtime	Supabase Realtime channels	Broadcast of accepted changes
Asset format	glTF 2.0 (GLB)	Prop library; authored in Autodesk Maya 2027
AI service (planned)	Depth model, segmentation, geometry solver	Optional later feature; not yet evaluated

3.10 System Module Descriptions

Table 3.6 System module descriptions

Module	Responsibility	Inputs / outputs
Authentication and roles	Sign-in, role lookup, role assignment	Credentials → session with role claim
Scene editor	Render and edit 3D objects	User gestures → candidate mutations
Properties inspector	Edit parameters by department	Selected object → updated properties
Concurrency manager	Check versions, reject stale edits, optional lease	Mutation + version → commit or reject
Review workflow	Track pending/approved/revision states	Object change → review record
Asset library	Store and serve glTF props	Upload → validated asset, LOD

Module	Responsibility	Inputs / outputs
Timeline view	Show production phases and milestones	Approved items → timeline state
AI layout service (planned)	Propose initial layout from photo	Photo + dimensions → parametric objects

3.11 Proposed Scene Mutation Structure

A scene change is transmitted as a compact structured message. The fields below are proposed for design and testing.

Table 3.7 Scene mutation message structure

Field	Purpose	Notes
object_id	Unique identifier of the scene object	UUID
project_id	Project the object belongs to	Used for channel routing
user_id	Author of the change	From the authenticated session
version	Client’s last known object version	Compared with the stored version
transform	Position, rotation (quaternion) and scale	Six degrees of freedom plus scale
properties	Department-specific parameters	Validated against the role
timestamp	Time of the change	For audit and ordering

The mutation procedure used by the concurrency manager is outlined below in simplified form. It is a design description, not a claim about the deployed code.

```
function ApplyMutation(M, S):
  O = S.FetchObject(M.object_id)
  if not RoleMayEdit(M.user_id, O.department): reject(403)
  if M.version != O.version:      reject(409, O.state, O.version)
  T_new = O.transform + M.delta
  if Overlaps(T_new, S.scene):    warn(overlap)
  O.transform = T_new; O.version += 1
  O.status = PENDING_REVIEW
  S.commit(O); broadcast(O); return success(O.version)
```

If an object receives repeated conflict rejections within a short window, the gateway may grant one user a short exclusive lease on it, rejecting other edits during the lease and releasing it after inactivity. The window and lease duration are parameters to be tuned by testing.

3.12 Conceptual Data Entities

Table 3.8 Conceptual data entities

Entity	Representative attributes	Purpose
Project	projectId, name, venueDims, createdBy	Top-level container for a stage plan

Entity	Representative attributes	Purpose
StageObject	objectId, department, transform, version, status	A placed element in the 3D scene
UserRole	userId, role, projectId	Maps users to department roles
ReviewRecord	reviewId, objectId, decision, annotations	Art Director decisions and comments
Asset	assetId, gltfUri, lodLevels, collisionBounds	A reusable 3D prop
TimelineMilestone	milestoneId, phase, lockedAt	A production phase gate
AuditLog	logId, userId, action, timestamp	Record of accepted and rejected changes

The conceptual model identifies the main information objects. These are logical design entities, not a claim that corresponding database tables or classes already exist with these names in the source code.

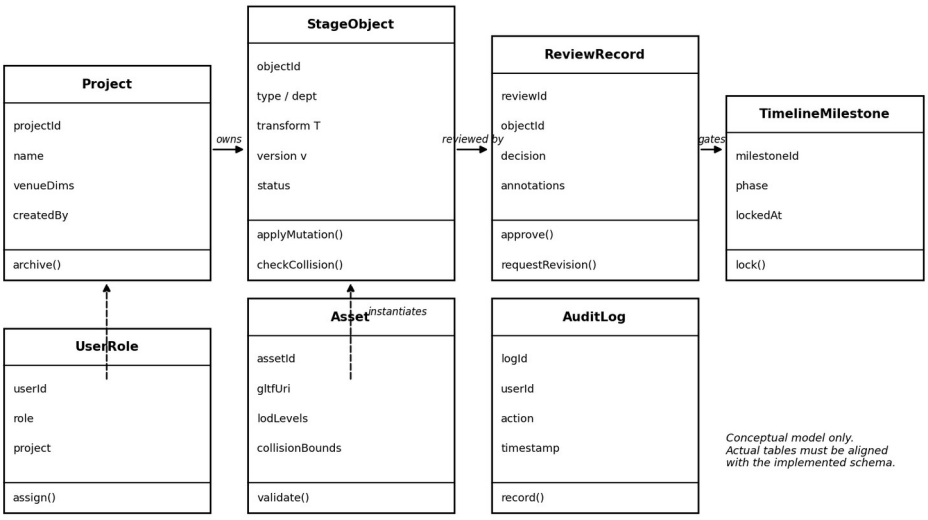


Figure 3.8 Core conceptual classes / entities

3.13 Venue-Aware AI Layout Design (Planned)

The proposed AI pipeline has five stages (Figure 3.9). The user uploads a venue photograph and enters the venue width, depth and height. A depth model and a segmentation model estimate relative depth and isolate the floor. Camera geometry is estimated from vanishing points, and the user-supplied dimensions fix the metric scale through a ground-plane homography. A constrained optimization then places a stage footprint on the floor, penalizing distance from the centre and the rear wall, favouring audience sightlines and enforcing clearance from boundaries for egress. The result is converted to editable parametric stage objects in the 3D scene, so that every department can refine it.

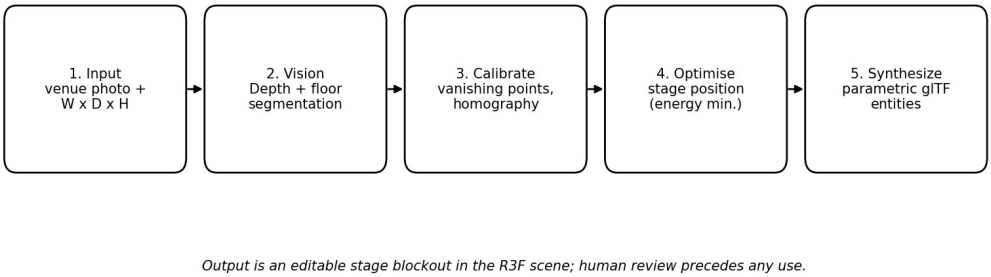


Figure 3.9 Venue-aware AI stage generation pipeline

This pipeline is a design proposal. Known risks include scale errors on wide-angle photographs, floor segmentation failures and over-simplified placement rules. Its output must always be reviewed by a person, and its accuracy must be measured before any benefit is claimed.

3.14 Planned Lighting and Acoustic Aids

As later enhancements, the 3D view could display simple lighting and sound estimates. For a lighting fixture, illuminance at a point may be estimated by the inverse-square law with a cosine factor, $E = [I(\theta) / d^2] \cdot \cos \theta$, where $I(\theta)$ is luminous intensity, d is distance and θ is incidence angle. Sound level over distance may be estimated using the attenuation terms of ISO 9613-2 [15]. These are planning aids only; they are not validated engineering calculations and are not part of the Phase I evaluation.

3.15 Security, Privacy, and Operational Design

Unreleased event designs and team information are sensitive. The design therefore requires authenticated sessions, encrypted connections, row-level security policies in the database and a clear separation between viewing rights and editing rights. Role assignment must be limited to the Admin. The system should store only the data needed for planning and should keep an audit log of accepted and rejected changes. Uploaded assets should be validated before entering the shared library. Free-tier services have usage limits, so deployment choices must be reviewed before real events depend on the tool.

3.16 Chapter Summary

The design chapter defines functional and non-functional requirements, architecture, DFD levels, use cases, activity flow, resources and roles, software modules, the mutation structure, data entities and the planned AI and safety considerations. The next chapter translates the design into a staged implementation and verification plan.

CHAPTER 4 — IMPLEMENTATION METHODOLOGY AND TESTING

4.1 Development Environment

The software stack comprises React and TypeScript for the web application, Three.js with React Three Fiber for the 3D viewport, and a managed Supabase back end with PostgreSQL for authentication, storage and real-time channels. 3D assets are authored in Autodesk Maya 2027 and exported to glTF. The exact framework versions, database schema, policies and deployment configuration must be verified from the project source files before being documented as facts.

4.2 Implementation Methodology

Table 4.1 Development and validation stages

Stage	Work item	Exit condition
1	Confirm requirements and design challenge (S.C.O.P.E.S)	Scope sheet approved by faculty incharge
2	Set up project, repository and back-end project	Application builds and connects to the database
3	Implement sign-in and role assignment	Each test user receives exactly one intended role
4	Implement 3D scene, transforms and grid snapping	Objects can be placed, moved and saved
5	Enforce role scope with database policies	A role cannot modify another department’s objects
6	Implement versioned mutations and real-time broadcast	Two clients stay consistent; stale edit is rejected
7	Implement review states and Art Director actions	Change sets pending; decision recorded
8	Add asset library and timeline view	Props load; milestones display
9	Prototype venue-aware AI layout	Photo and dimensions produce an editable layout
10	Run evaluation and usability study	Metrics recorded reproducibly

4.3 Known Development Status

The following status reflects only what is documented at the time of report preparation. Items marked “to be verified” must be confirmed against the source code and tests before they are claimed.

Table 4.2 Known development status

Item	Current evidence	Status
Design challenge and scope	S.C.O.P.E.S sheet dated 30/07/2026	Completed
Research paper draft	Framework paper prepared by the team	Completed
Technology stack	Selected and documented	Defined

Item	Current evidence	Status
Account sign-in and role table	Per-user accounts and a roles table intended	In progress; to be verified
Per-role edit restriction	Required by design; enforcement not yet confirmed	To be verified
3D editor and property inspector	Described in the design	To be verified from source
Versioned mutations / real-time sync	Designed (Section 3.11)	Planned; to be verified
Art Director review workflow	Designed (Figure 4.1)	Planned; to be verified
AI venue-aware layout	Designed (Section 3.13)	Planned; not implemented
Performance and usability results	None recorded	Not measured

4.4 Implementation Approach for Core Modules

4.4.1 Authentication and Role Enforcement

Each user signs in with an e-mail account. A roles table maps the user to one department role. Permissions are enforced by row-level security policies in PostgreSQL that compare the authenticated user's role with the department of the object being changed, so that a direct request to the database cannot bypass the interface. The corresponding test (T-03) must show that an Art Director can approve but not move lighting fixtures, and that a Lighting Lead cannot modify stage decks.

4.4.2 Real-Time Synchronization and Concurrency

When a user transforms an object, the client updates the local scene immediately and sends the change with the object's last known version. The server accepts the first valid change for that version, increments the version and broadcasts the result. A client whose change was rejected receives the authoritative state and smoothly moves the object back to it. If an object is repeatedly contested, a temporary lease may be used. These behaviours are covered by tests T-05 and T-06.

4.4.3 Review and Approval Workflow

A scene object has a governance status. A new or edited object becomes pending review; the Art Director may approve it, or request revision with a comment or a pin annotation on the 3D model. Any later edit returns the object to pending review. Approved items can then be locked into a production timeline phase.

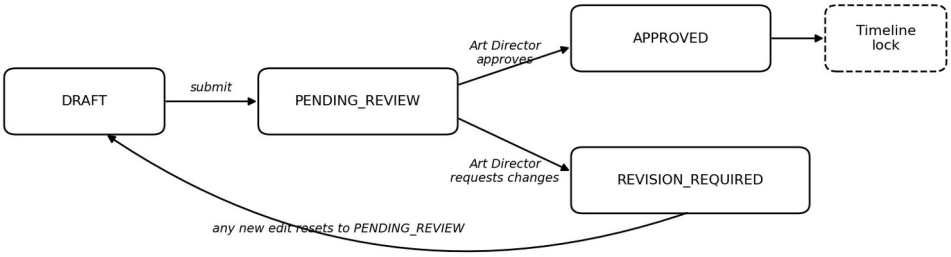


Figure 4.1 Governance state machine for stage objects

4.5 Application Interface Implementation

The interface is designed so that each role sees the same 3D scene but only the controls relevant to its work. It includes a sign-in screen, a project dashboard, a 3D editor with a properties inspector, a review panel for the Art Director and a timeline view. Screenshots from the running application should be inserted in the placeholders below before submission.

4.5.1 Approval Page Dashboard

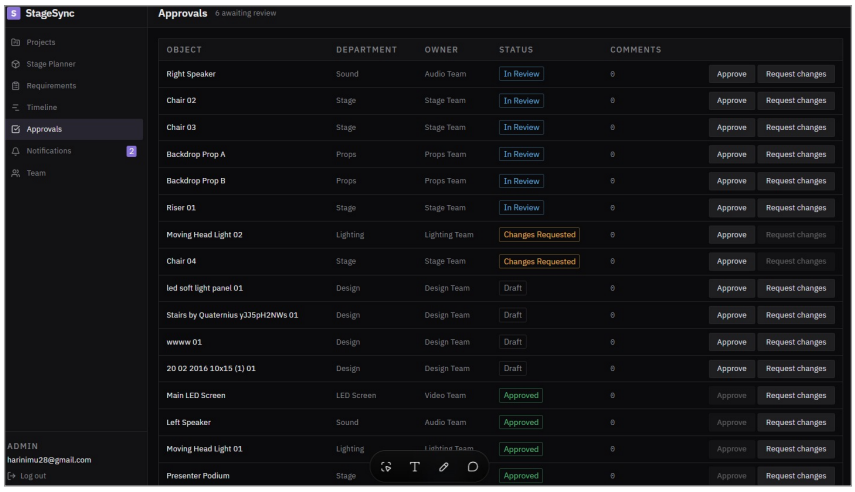


Figure 4.2 Approval page dashboard

4.5.2 3D Stage Editor

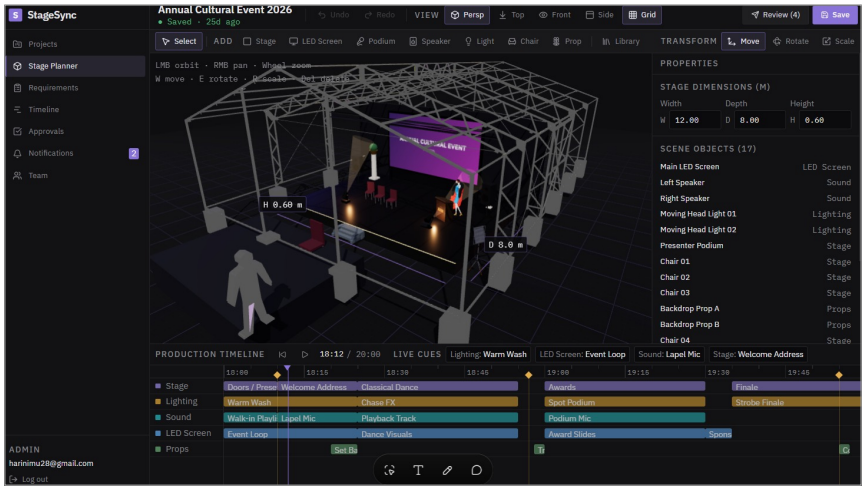


Figure 4.3 3D stage editor with properties inspector

4.5.3 Interface Implementation Status

Screenshots document the appearance and available controls of the interface. They do not by themselves establish that role restrictions, synchronization, review workflow or AI layout work correctly. These functions must be validated using the test cases in Section 4.6.

4.6 Test Strategy

Testing proceeds from component checks to integration and then to controlled measurement. A failed prerequisite blocks interpretation of later tests; for example, synchronization latency is not meaningful until two clients can edit the same scene consistently.

4.6.1 Test Cases

Table 4.3 Test cases and current status

Test ID	Test objective	Expected result	Actual result / status
T-01	Sign-in and role assignment	User signs in and receives the intended role	Not yet executed
T-02	Scene create/move/save	Object persists after reload	Not yet executed
T-03	Role restriction	Out-of-scope edit is rejected by the database	Not yet executed
T-04	Real-time broadcast	Second client sees the change	Not yet executed
T-05	Stale-version edit	Older version is rejected; client reverts	Not yet executed
T-06	Repeated contention	Lease limits rejection loops	Not yet executed
T-07	Review workflow	Edit sets pending; decision recorded	Not yet executed
T-08	Overlap warning	Overlapping objects trigger a warning	Not yet executed
T-09	Asset upload and load	glTF prop loads; invalid file rejected	Not yet executed
T-10	AI layout (planned)	Photo and dimensions yield an editable layout	Not yet executed

4.6.2 Performance Metrics

Table 4.4 Performance metrics

Metric	Definition / formula	Required record
Synchronization latency	Remote client update time − local commit time	Clock basis, network, client count
Viewport frame rate	Frames per second during standard interaction	Device, browser, object count
Conflict rejection rate	Rejected edits ÷ total edits	Number of users, contention pattern

Metric	Definition / formula	Required record
Planning turnaround time	Time from brief to approved stage plan	Team size, tool used, task description
Revision cycles	Number of review rounds before approval	Teams, task, tool used
Conflicts caught before event	Overlaps or clashes found in planning	Detection method, event type
Usability score	Questionnaire result from participants	Questionnaire, participant roles

4.6.3 Performance Measurement Template

The evaluation is planned as a comparison between the team's current practice (messages, documents and sketches) and StageSync, using small teams of student event volunteers. The table below fixes the trial design (teams, tools and tasks); the result columns are to be filled only from real measurements, and no results are claimed.

Table 4.5 Performance measurement template

Trial	Users / teams	Tool	Task	Time taken	Revision cycles	Conflicts found	Notes
1	Team A	Baseline	Plan a 3-team stage set-up	To record	To record	To record	Messages, documents, sketches
2	Team A	StageSync	Same task in the shared 3D scene	To record	To record	To record	After short orientation
3	Team B	StageSync	Plan a different stage layout	To record	To record	To record	Order swapped
4	Team B	Baseline	Same task as trial 3	To record	To record	To record	Messages, documents, sketches

4.7 Discussion of Limitations

The main technical risks are correct database-level enforcement of department permissions, handling of simultaneous edits under contention, and performance of the 3D viewport on lower-end devices. Free-tier hosting limits may also constrain real-time usage. The AI layout feature depends on photograph quality and on correct user-supplied dimensions. Not all team members may be equally comfortable adopting a new digital tool under time pressure, so training time and interface simplicity must be part of the usability evaluation.

4.8 Ethical, Safety, and Operational Considerations

StageSync is a planning aid. Stage structures, rigging and electrical systems have safety implications that must be assessed by qualified professionals, and layouts proposed by the tool, including any AI-generated layout, must be reviewed by people before use. Participant data collected during evaluation should be minimal, collected with consent and stored

securely. Venue photographs and event designs may be confidential and should be accessible only to authorized project members.

4.9 Chapter Summary

This chapter establishes a staged implementation plan, records the known development status conservatively, and provides test cases, metrics and measurement templates. The next phase should first confirm role enforcement and synchronization before expanding to the review workflow, timeline and AI layout features.

CHAPTER 5 — CONCLUSION AND FUTURE ENHANCEMENTS

5.1 Conclusion

StageSync addresses the design challenge of coordinating stage, props, lighting and sound teams and enabling the Art Director to review and approve stage requirements before an event. The proposed system combines a shared browser-based 3D scene, department-level role-based access control, versioned real-time synchronization, an Art Director review workflow, a production timeline view, and an optional venue-aware AI layout feature. This report defines the requirements, architecture, DFD levels, use cases, activity flow, resources and roles, module and data design, and the testing approach needed to develop the platform systematically.

The evidence currently available supports a limited status statement: the design challenge, architecture and test plan are defined, while the behaviour of each module remains to be verified against the source code and tests. No synchronization latency, frame rate, usability or AI accuracy results are claimed. The next step is to confirm role enforcement and real-time synchronization and then to execute the test plan in sequence.

5.2 Current Phase-I Status

- Design challenge framed and scoped using the S.C.O.P.E.S sheet.
- Stakeholders, objectives and outcomes documented.
- Literature background and comparison with existing approaches prepared.
- Overall architecture, DFD Level 0, 1 and 2, use-case, activity, functional and class diagrams prepared.
- Role-based access matrix, mutation structure and review workflow designed.
- Test cases, metrics and blank measurement templates prepared.

5.3 Limitations

- Module behaviour has not yet been confirmed against tests; some features are designed but not demonstrated.
- No measured performance, scalability or usability data are available.
- The AI layout pipeline is a proposal with unknown accuracy on real venue photographs.
- Offline operation, mobile optimization, structural load analysis and formal security review are not addressed.

5.4 Future Enhancements

Table 5.1 Future enhancement plan

Area	Proposed enhancement	Expected benefit
Role security	Complete and test database policies for every role	Reliable department-level restrictions
Concurrency	Tune conflict window and lease behaviour from test data	Fewer rejected edits under contention

Area	Proposed enhancement	Expected benefit
Review workflow	Spatial annotations and review history view	Clearer feedback for the Art Director
Timeline	Integrate calendar and notifications	Shared schedule across teams
AI layout	Implement and measure photo-to-layout pipeline	Faster initial stage blockout
Simulation aids	Add simple lighting and sound estimates	Early detection of coverage problems
Offline support	Cache assets and queue edits	Use at venues with poor connectivity
Evaluation	Run controlled studies with event teams	Evidence of coordination benefit

5.5 Proposed Phase-II Work Schedule

Table 5.2 Proposed Phase-II work schedule

Phase	Planned activity	Expected outcome
Phase II-1	Verify current code; confirm sign-in and roles	Documented, verified baseline
Phase II-2	Database-level role enforcement and tests	Role restrictions demonstrated
Phase II-3	Versioned mutations and real-time sync	Consistent multi-client editing
Phase II-4	Review workflow and timeline integration	Traceable approval flow
Phase II-5	AI layout prototype and asset pipeline	Editable layout from a photograph
Phase II-6	Evaluation with event teams; final report	Documented metrics and limitations

5.6 Final Remarks

StageSync should be assessed on repeatable evidence rather than architectural intent alone. A reliable demonstration requires verified role enforcement, consistent synchronization, a working review workflow, and a measured comparison with current practice. These steps provide a defensible path from the current design to a tested prototype.

REFERENCES

[1] Three.js Authors, “Three.js Documentation.” Available at: https://threejs.org/docs/

[2] Poimandres, “React Three Fiber Documentation.” Available at: https://docs.pmnd.rs/react-three-fiber

[3] Meta Platforms, “React Documentation.” Available at: https://react.dev

[4] Supabase Inc., “Supabase Documentation” (Auth, Realtime, Row Level Security). Available at: https://supabase.com/docs

[5] PostgreSQL Global Development Group, “PostgreSQL Documentation: Row Security Policies.” Available at: https://www.postgresql.org/docs/

[6] Khronos Group, “glTF 2.0 Specification.” Available at: https://www.khronos.org/gltf/

[7] H. T. Kung and J. T. Robinson, “On optimistic methods for concurrency control,” ACM Trans. Database Syst., vol. 6, no. 2, pp. 213–226, 1981.

[8] C. A. Ellis and S. J. Gibbs, “Concurrency control in groupware systems,” ACM SIGMOD Rec., vol. 18, no. 2, pp. 399–407, 1989.

[9] M. Shapiro, N. Preguiça, C. Baquero and M. Zawirski, “Conflict-free replicated data types,” in Proc. SSS, 2011, pp. 386–400.

[10] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler and V. Koltun, “Towards robust monocular depth estimation: Mixing datasets for zero-shot cross-dataset transfer,” IEEE Trans. Pattern Anal. Mach. Intell., vol. 44, no. 3, pp. 1623–1637, 2022.

[11] R. Ranftl, A. Bochkovskiy and V. Koltun, “Vision transformers for dense prediction,” in Proc. IEEE/CVF ICCV, 2021, pp. 12179–12188.

[12] A. Kirillov et al., “Segment Anything,” in Proc. IEEE/CVF ICCV, 2023, pp. 4015–4026.

[13] D. G. Lowe, “Distinctive image features from scale-invariant keypoints,” Int. J. Comput. Vis., vol. 60, no. 2, pp. 91–110, 2004.

[14] G. Bradski, “The OpenCV Library,” Dr. Dobb’s Journal of Software Tools, vol. 25, no. 11, pp. 120–125, 2000.

[15] International Organization for Standardization, “ISO 9613-2: Acoustics — Attenuation of sound during propagation outdoors — Part 2: General method of calculation,” 1996.

[16] Autodesk Inc., “Maya Documentation.” Available at: https://help.autodesk.com/

[17] ESTA, “ANSI E1.11: Entertainment Technology — USITT DMX512-A, Asynchronous Serial Digital Data Transmission Standard,” 2018.

[18] Department of Computer Science and Design, Rajalakshmi Engineering College, “S.C.O.P.E.S – Team Scope Sheet: StageSync,” GE23627 Design Thinking and Innovation, AY 2026–2027, 30 July 2026.

[19] A. R and H. M., “StageSync: A collaborative 3D stage-planning framework with multi-role coordination and venue-aware AI spatial synthesis,” project research paper, Dept. of Computer Science and Design, Rajalakshmi Engineering College, 2026.

Reference integrity note: The entries above identify documentation, established standards and published papers. Before final submission, verify the exact document version, access date and applicability of each source, and add further peer-reviewed literature on collaborative 3D environments and event production only after confirming its bibliographic details.

APPENDIX A — TOOLS AND ENVIRONMENT CHECKLIST

Table A.1 Tools and environment checklist

Item	Purpose	Checked (Y/N)	Notes
Node.js and package manager	Build the web application	Pending	Record versions
React / TypeScript project	Front end	Pending	Record framework and versions
Three.js / React Three Fiber	3D viewport	Pending	Record versions
Supabase project	Auth, database, realtime	Pending	Record region and plan limits
Autodesk Maya 2027	Authoring props	Pending	Record export settings to glTF
Test devices and browsers	Frame-rate and usability tests	Pending	Record models and browsers

APPENDIX B — MUTATION EXCHANGE TRACE TEMPLATE

The table fixes the planned trace scenarios for tests T-04 to T-06; time and result columns are to be recorded during testing. No trace data are claimed.

Table B.1 Mutation exchange trace template

Mutation ID	Time	User / role	Object	Version sent	Result	Notes
MUT-001	To record	Stage Design	Deck 1	1	To record	Baseline accepted edit (T-04)
MUT-002	To record	Lighting Lead	Truss 1	1	To record	Concurrent edit, first arrival (T-05)
MUT-003	To record	Props Team	Truss 1	1	To record	Stale-version edit; expect reject (T-05)
MUT-004	To record	Lighting Lead	Fixture 2	2	To record	Contention and lease test (T-06)

APPENDIX C — S.C.O.P.E.S TEAM SCOPE SHEET SUMMARY

Team: StageSync (Abhishek R, Harini M). Domain: EventTech (Collaboration & Production Planning). Date: 30/07/2026. Faculty incharge: Dr. P. Revathy. Course: GE23627 Design Thinking and Innovation, AY 2026–2027.

Table C.1 SCOPES summary of the design challenge

Element	Summary
S – Situation / Problem	Event teams (stage, props, lighting, sound) communicate through separate messages, documents, sketches and discussions, causing miscommunication, unclear responsibilities and last-minute changes, commonly in college events and cultural programmes.
C – Constraints	Separate communication between teams; unclear responsibilities; frequent last-minute changes; difficulty reviewing the whole stage plan in one place; varied tool habits; short timelines; free or low-cost tools only; uneven comfort with new tools; difficulty representing a physical layout digitally.
O – Objectives and outcomes	Centralize requirements in a shared visual plan; clarify ownership; surface changes early; give the Art Director a review and approval workflow; provide a shared timeline; spend less time chasing updates; fewer conflicts and surprises.
P – People	Primary: Art Director, Event Organizer, Props and Stage Design, Lighting, Sound. Secondary: Stage Managers, Event Coordinators, Production Crew. Decision makers: Art Director, Event Head.
E – Estimates	Figma/Miro, React/Next.js, Node.js + Express, MongoDB/PostgreSQL, Firebase, Google Calendar API, AWS/GCP (free tiers or student credits).
S – Scope to explore	Visual stage planning, department-wise collaboration, feedback and approval workflows, timeline coordination, and different ways to present stage requirements.
Design challenge	How might we improve coordination between different event production teams and enable the Art Director to effectively review, provide feedback and approve stage-related requirements before the event?